



Fecha: 07-09-2026

ACIF104

Sumativa 3: Informe Final del Proyecto.

“Clasificación del Segmento de Precio de Vehículos Usados”.

Nombre:

Gabriel Jara Rivas.
Matías Lillo Yévenes.
Nicolás Duarte Maldonado.
Nicolás Navarrete Caro.

Curso: 202615.2404 | APRENDIZAJE DE MÁQUINA | ACIF104.

Índice

- I. Problemática.....3**
 - 1.1. Contexto y relevancia3
 - 1.2. Formulación del problema.....3
 - 1.3. Requisitos iniciales del proyecto.....4
- II. Metodología5**
 - 2.1. Fases metodológicas.....5
 - 2.2. Plan de trabajo.....5
- III. Desarrollo del proyecto7**
 - 3.1. Análisis exploratorio de datos7
 - 3.2. Selección de técnicas candidatas.....8
 - 3.3. Comparación de técnicas.....9
 - 3.3.1. Validación bajo un protocolo de balanceo homogéneo9
 - 3.4. Requisitos del proyecto10
 - 3.5. Selección de la arquitectura12
 - 3.6. Refinamiento de Random Forest y balanceo.....12
 - 3.7. Eliminación de la variable redundante age13
 - 3.8. Desarrollo del frontend y backend14
- IV. Resultados16**
 - Interpretabilidad: SHAP sobre el modelo final17
- V. Conclusiones, limitaciones y trabajos futuros.....18**
 - 5.1. Reflexión crítica sobre el proceso de consolidación.....18
 - 5.2. Limitaciones18
 - 5.3. Propuesta de mejoras.....19
 - 5.4. Conclusión.....19
- VI. Código fuente en GitHub.....20**
- VII. Bibliografía21**

I. Problemática

En esta sección se describe la problemática que motiva el proyecto, se analiza su relevancia a partir de antecedentes recientes y se formula como una tarea de clasificación multiclase. Además, se establecen los requisitos iniciales y el alcance de la solución propuesta.

1.1. Contexto y relevancia

El mercado de vehículos usados constituye un entorno de alta complejidad para la determinación de precios. A diferencia de los vehículos nuevos, cuyo valor inicial es establecido por el fabricante, el precio de un vehículo usado depende de atributos heterogéneos —antigüedad, kilometraje, condición, marca, carrocería y equipamiento— que interactúan de forma no lineal. Esta situación genera incertidumbre para compradores y vendedores, además de asimetrías de información que dificultan reconocer publicaciones sobrevaloradas o subvaloradas.

La literatura reciente demuestra que el aprendizaje automático puede mejorar la estimación del valor de reventa, aunque sus resultados deben interpretarse considerando las características de cada estudio. Bergmann y Feuerriegel (2025), por ejemplo, obtuvieron mejoras al incorporar información granular sobre el equipamiento del vehículo en un conjunto de 92.239 transacciones. Sin embargo, ese resultado no es directamente transferible al presente proyecto, ya que `vehicles_us.csv` contiene información más limitada sobre equipamiento y proviene exclusivamente del mercado estadounidense. En consecuencia, su estudio respalda la relevancia de enriquecer los atributos disponibles, pero también advierte que la capacidad predictiva depende de la cobertura y calidad de los datos.

Li (2024) y Qian (2023) reportan buenos resultados mediante Random Forest y XGBoost en la predicción de precios de vehículos usados. No obstante, ambos trabajos formulan principalmente el problema como regresión y evalúan el desempeño mediante métricas como el coeficiente de determinación. El presente proyecto adopta una formulación diferente: clasificación multiclase por segmento de precio. Por ello, esos antecedentes justifican la selección inicial de modelos basados en árboles, pero sus métricas no son directamente comparables con el F1-macro utilizado en este estudio.

Esta elección también es coherente con el análisis comparativo de Grinsztajn, Oyallon y Varoquaux (2022), quienes evaluaron modelos neuronales y modelos basados en árboles en múltiples conjuntos tabulares y concluyeron que estos últimos continúan siendo especialmente competitivos en datos tabulares de tamaño medio. Sin embargo, su estudio no aborda específicamente el mercado automotor. En este proyecto, su conclusión se considera una hipótesis por contrastar, no una garantía de que Random Forest o XGBoost superarán necesariamente a las redes neuronales en `vehicles_us.csv`. Como se detalla en la sección III.3, esta hipótesis fue puesta a prueba nuevamente bajo un protocolo de comparación homogéneo, con el mismo resultado.

La distribución de la variable objetivo presenta además un desbalance importante entre los tres segmentos. Chen et al. (2024) señalan que no existe una única técnica de balanceo superior en todos los contextos y que su efectividad depende de la distribución y del clasificador utilizado. Por esta razón, el proyecto compara datos sin balanceo, ponderación de clases, SMOTE y submuestreo aleatorio dentro del proceso de entrenamiento, sin aplicar remuestreo sobre los conjuntos de validación y prueba. De este contraste se desprende que la relevancia del proyecto no se limita a obtener un modelo con buen desempeño: también busca identificar las variables asociadas a cada segmento, controlar el efecto del desbalance y reconocer los límites de generalización a otros mercados o periodos.

1.2. Formulación del problema

A partir del conjunto de datos `vehicles_us.csv`, compuesto originalmente por 51.525 avisos de vehículos usados publicados en Estados Unidos, se plantea una tarea de clasificación multiclase: dados los atributos disponibles de un vehículo, predecir el segmento de precio al que pertenece.

Antes de construir la variable objetivo se aplica el proceso de limpieza definido en `preprocessing.py`. Este elimina precios inferiores a USD 500 o superiores a USD 300.000, descarta años de fabricación anteriores a 1980 e imputa valores faltantes o inválidos en variables como año, kilometraje, cilindros, tracción y color. Como resultado, el conjunto utilizado para el modelado queda compuesto por 50.267 registros.

La variable objetivo se define mediante dos umbrales: Económico para precios inferiores a USD 5.000, Medio para precios entre USD 5.000 y USD 20.000, y Premium para precios superiores a USD 20.000. La distribución resultante es de 11.746 vehículos Económicos (23,4 %), 29.856 vehículos del segmento Medio (59,4 %) y 8.665 vehículos Premium (17,2 %).

Esta distribución confirma un desbalance relevante, debido principalmente al predominio del segmento Medio. En consecuencia, la exactitud global no resulta suficiente para evaluar los modelos, ya que podría ocultar un desempeño deficiente en las clases menos representadas. La pregunta rectora se formula así: ¿qué características de un vehículo usado permiten clasificarlo en un segmento de precio y qué estrategia de modelado ofrece el mejor equilibrio de desempeño entre las tres clases?

1.3. Requisitos iniciales del proyecto

- **Objetivo funcional:** clasificar un vehículo en uno de los tres segmentos definidos mediante los umbrales de USD 5.000 y USD 20.000, utilizando sus atributos técnicos y comerciales.
- **Explicabilidad:** identificar globalmente las variables asociadas con las predicciones del modelo y presentar al usuario factores comprensibles derivados del análisis exploratorio y de explicabilidad.
- **Manejo del desbalance:** comparar distintas técnicas de balanceo y mantener un desempeño adecuado en las clases menos representadas, evitando la contaminación de los conjuntos de validación y prueba.
- **Operacionalización:** exponer la solución mediante una interfaz web y una API que permitan generar predicciones, visualizar las probabilidades estimadas y monitorear el funcionamiento básico del servicio.
- **Métricas de evaluación:** utilizar F1-macro como métrica principal, debido al desbalance entre los segmentos, complementada con accuracy, precisión, recall por clase y AUC multiclase One-vs-Rest con promedio macro.
- **Alcance:** desarrollar una solución experimental aplicable al conjunto de datos `vehicles_us.csv`, reconociendo que sus resultados corresponden al mercado estadounidense y que no pueden generalizarse directamente a otros países, periodos o fuentes de información sin una nueva validación.

II. Metodología

El proyecto se desarrolló siguiendo una metodología iterativa basada en las fases clásicas de un proyecto de ciencia de datos (Géron, 2022), incorporando en su etapa final un proceso de verificación y consolidación (semanas 10 y 11) orientado a validar empíricamente decisiones de diseño que en una primera instancia habían quedado únicamente declaradas como limitaciones metodológicas.

2.1. Fases metodológicas

- **Análisis exploratorio de datos (EDA):** caracterización de las variables, evaluación de calidad (completitud, consistencia y correctitud), detección de valores atípicos y estudio de correlaciones y patrones.
- **Selección de técnicas:** elección fundamentada de algoritmos de aprendizaje supervisado, incluyendo modelos clásicos de machine learning y arquitecturas de deep learning.
- **Diseño de la arquitectura:** definición de la ingeniería de características, la codificación de variables y la topología de las redes neuronales.
- **Implementación, validación y evaluación:** entrenamiento con partición estratificada train/validación/test, análisis de técnicas de balanceo, evaluación con datos no vistos e interpretación mediante SHAP.
- **Verificación y consolidación final:** reejecución de la comparación ML/DL bajo un protocolo de balanceo homogéneo, eliminación de variables redundantes, recálculo de SHAP sobre el modelo final real y migración de la interfaz de reglas heurísticas a explicaciones locales.

2.2. Plan de trabajo

La Tabla 1 vincula cada tarea con una persona responsable, un plazo y el objetivo del proyecto al que contribuye, incluyendo las tareas de verificación y consolidación final (semanas 10 a 12).

Tarea	Responsable	Plazo	Objetivo asociado
Limpieza y análisis exploratorio de datos	Gabriel Jara	Semana 1	Evaluar calidad de datos, variables y patrones
Ingeniería de características	Matías Lillo	Semana 1	Preparar las variables utilizadas por los modelos
Modelos de ML y comparación	Nicolás Duarte	Semana 2	Seleccionar técnicas tradicionales de clasificación
Modelos de DL y análisis de convergencia	Nicolás Navarrete	Semana 2	Comparar arquitecturas neuronales con modelos tradicionales
Evaluación de técnicas de balanceo	Gabriel Jara	Semana 3	Mejorar el desempeño en clases minoritarias
Análisis de explicabilidad global con SHAP	Matías Lillo	Semana 3	Identificar variables asociadas con las predicciones
Desarrollo inicial del frontend y backend	Nicolás Duarte	Semana 3	Operacionalizar el modelo mediante interfaz web y API
Redacción y revisión de la primera entrega	Todo el equipo	Semana 4	Entregable Sumativa 1
Análisis de retroalimentación y auditoría de resultados	Todo el equipo	Semana 5	Identificar inconsistencias entre informe, código y resultados

Tarea	Responsable	Plazo	Objetivo asociado
Regeneración del EDA y corrección de la variable objetivo	Gabriel Jara	Semana 5	Alinear la Figura 1, conteos y porcentajes con el pipeline real
Refinamiento y barrido de hiperparámetros de Random Forest	Nicolás Duarte	Semana 6	Justificar cantidad de árboles, profundidad, tiempo y tamaño
Corrección de callbacks y validación de modelos DL	Nicolás Navarrete	Semana 6	Evitar la reutilización de estado de EarlyStopping
Revisión del análisis SHAP y variables redundantes	Matías Lillo	Semana 7	Declarar el uso del modelo sustituto y la redundancia model_year/age
Actualización del frontend, backend y monitoreo	Nicolás Duarte	Semana 7	Diferenciar probabilidades, orientación heurística y explicabilidad
Reentrenamiento y evaluación reproducible del modelo final	Gabriel Jara	Semana 8	Generar métricas, matriz de confusión y modelo comprimido
Actualización de scripts, dependencias y README	Nicolás Navarrete	Semana 8	Garantizar instalación, ejecución y trazabilidad del repositorio
Validación cruzada entre código, figuras e informe	Todo el equipo	Semana 9	Comprobar correspondencia de métricas, tablas, figuras y conclusiones
Redacción y revisión de la Sumativa 2	Todo el equipo	Semana 9	Entregable Sumativa 2
Comparación homogénea ML/DL con SMOTE uniforme	Nicolás Navarrete	Semana 10	Controlar el tratamiento del desbalance entre familias de modelos
Eliminación de la variable redundante age	Gabriel Jara	Semana 10	Corregir la distorsión de importancia en el análisis SHAP
Recálculo de SHAP sobre el modelo final real y explicación local en la API	Matías Lillo	Semana 11	Reemplazar el modelo sustituto y las reglas heurísticas por atribuciones locales reales
Reentrenamiento del modelo final, pruebas end-to-end de la API y actualización del frontend	Nicolás Duarte	Semana 11	Verificar que las mejoras no degraden el desempeño ni la operación del sistema
Redacción y revisión final del informe integrado	Todo el equipo	Semana 12	Entregable Sumativa 3 — Informe final

Tabla 1. Plan de trabajo grupal.

III. Desarrollo del proyecto

En esta sección se presenta el desarrollo técnico del proyecto: el análisis y preparación de los datos, la comparación de las técnicas candidatas —incluyendo su validación bajo un protocolo de balanceo homogéneo—, la selección y refinamiento del modelo, la resolución de la redundancia entre variables, y la implementación de la solución mediante una aplicación web con explicabilidad local real.

3.1. Análisis exploratorio de datos

El archivo original contiene 51.525 registros y 13 variables. Las variables numéricas son price, model_year, cylinders, odometer y days_listed; las variables categóricas son model, condition, fuel, transmission, type y paint_color. Además, is_4wd se trata como una variable binaria y date_posted como una variable temporal. Después de aplicar el pipeline de limpieza, el conjunto utilizado para el modelado contiene 50.267 observaciones.

Compleitud, consistencia y correctitud. En los datos originales se observaron ausencias en is_4wd (50,4 %), paint_color (18,0 %), odometer (15,3 %), cylinders (10,2 %) y model_year (7,0 %). La ausencia de is_4wd se interpretó como no 4x4; paint_color se transformó en la categoría unknown y las variables numéricas se imputaron mediante la mediana por modelo y, cuando fue necesario, la mediana global. También se filtraron precios fuera del intervalo USD 500–300.000 y años anteriores a 1980; odómetros iguales a cero o superiores a 400.000 se trataron como inválidos antes de imputar. El resultado no conserva valores faltantes en las variables empleadas por el modelo.

Estadísticas descriptivas. La Tabla 2 resume tendencia central, dispersión y rango en el dataset limpio. El precio mantiene un sesgo positivo: la media (USD 12.353) supera a la mediana (USD 9.500), mientras el máximo de USD 300.000 confirma una cola derecha extensa.

Variable	Media	Desv. estándar	Mínimo	Mediana	Máximo
Precio (USD)	12.353,12	9.824,64	500	9.500	300.000
Año del modelo	2.009,84	5,53	1.980	2.011	2.019
Cilindros	6,1	1,64	3	6	12
Odómetro	116.884,89	57.834,92	1	117.434,50	400.000
Días de publicación	39,56	28,21	0	33	271

Tabla 2. Estadísticas descriptivas del conjunto de datos después de la limpieza.

Variables categóricas. La Tabla 3 caracteriza su cardinalidad y categoría modal. Fuel y transmission están fuertemente concentradas en gas y automatic, respectivamente, mientras type y paint_color presentan mayor diversidad. Model posee 100 categorías, lo que justifica el uso de codificación one-hot en los modelos clásicos y embeddings en una de las redes.

Variable	Cardinalidad	Categoría modal	Frecuencia	%
model	100	ford f-150	2.724	5,42
condition	6	excellent	23.793	47,33
fuel	5	gas	46.048	91,61
transmission	3	automatic	46.034	91,58
type	13	SUV	12.095	24,06
paint_color	13	white	9.816	19,53

Tabla 3. Cardinalidad y categoría modal de las variables categóricas.

Variables binaria y temporal. Después de la limpieza, la variable `is_4wd` presenta 24.815 vehículos con tracción 4x4 (49,37 %) y 25.452 sin tracción 4x4 (50,63 %). Esta distribución prácticamente equilibrada indica que la tracción se encuentra suficientemente representada en ambos grupos y puede utilizarse como característica predictiva sin requerir un tratamiento especial por desbalance. Por su parte, `date_posted` abarca desde el 1 de mayo de 2018 hasta el 19 de abril de 2019, correspondiente a un periodo de 353 días. La cobertura temporal es inferior a un año completo, por lo que el modelo representa las condiciones del mercado observadas durante ese intervalo y no permite evaluar cambios de precio de largo plazo.

Valores atípicos y patrones. El precio se relaciona positivamente con `model_year` ($r = 0,52$) y negativamente con `odometer` ($r = -0,45$); `days_listed` no presenta asociación lineal relevante. La Figura 1 emplea el mismo dataset limpio y los mismos cortes del target que el pipeline: Económico, 11.746 casos (23,4 %); Medio, 29.856 (59,4 %); y Premium, 8.665 (17,2 %).

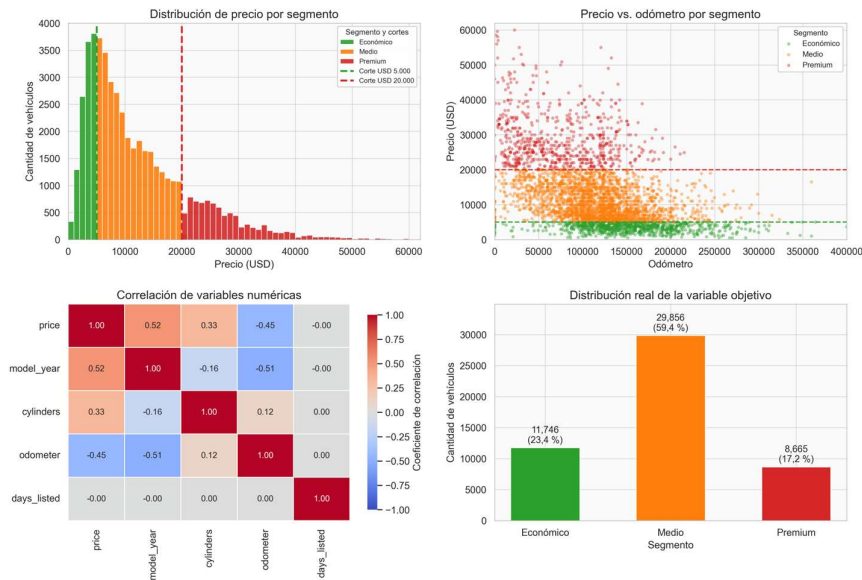


Figura 1. Panorama del análisis exploratorio después de la limpieza: distribución del precio, relación precio-odómetro, correlaciones y distribución real del target.

3.2. Selección de técnicas candidatas

La selección de técnicas se fundamentó en la revisión de trabajos previos. Para los datos tabulares y heterogéneos de este problema, la literatura converge en que los métodos basados en árboles suelen ser los más efectivos. Breiman (2001) introdujo el Random Forest, un ensamble de árboles que reduce la varianza mediante agregación; Chen y Guestrin (2016) propusieron XGBoost, un algoritmo de potenciación del gradiente que domina numerosas competencias de datos estructurados. Diversos estudios de predicción de precios de autos usados (Li, 2024; Qian, 2023; Bergmann y Feuerriegel, 2025) reportan a estos modelos como los de mejor desempeño.

Por exigencia del proyecto se incorpora además un modelo neuronal de aprendizaje profundo. Sin embargo, la evidencia reciente advierte que en datos tabulares las redes neuronales no siempre superan a los árboles: Grinsztajn et al. (2022) y Shwartz-Ziv y Armon (2022) demuestran empíricamente que los modelos basados en árboles mantienen ventaja en la mayoría de los conjuntos tabulares, debido a sus sesgos inductivos frente a características heterogéneas. Se incorpora el perceptrón multicapa (MLP) precisamente para contrastar esta hipótesis en el contexto del proyecto. Las técnicas seleccionadas son, por tanto: Regresión Logística (baseline lineal, interpretable), Random Forest (ensamble de árboles, robusto ante outliers),

XGBoost (potenciación del gradiente, alto rendimiento) y redes neuronales MLP (tres arquitecturas de deep learning, ver sección 3.5).

3.3. Comparación de técnicas

Se compararon tres técnicas de machine learning y tres arquitecturas de deep learning sobre el conjunto de validación, empleando F1-macro como métrica principal por su robustez frente al desbalance. La Tabla 4 y la Figura 2 resumen los resultados de esta primera comparación exploratoria.

Modelo	Tipo	Accuracy	F1-macro
Regresión Logística	ML	0,758	0,751
Random Forest	ML	0,862	0,846
XGBoost	ML	0,849	0,831
MLP Shallow	DL	0,805	0,797
MLP Profunda	DL	0,805	0,797
MLP Embeddings	DL	0,809	0,801

Tabla 4. Resumen de resultados de la comparación de F1-macro entre técnicas de Machine Learning y Deep Learning (protocolo original, balanceo mixto).

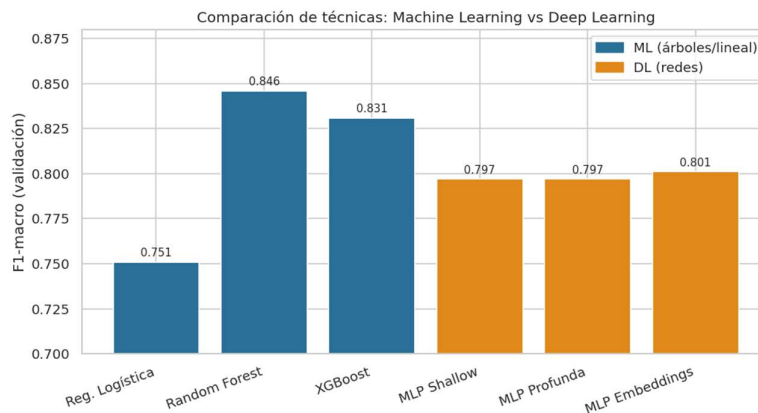


Figura 2. Comparación de F1-macro entre técnicas de Machine Learning y Deep Learning (protocolo original).

Los resultados confirman la hipótesis planteada: Random Forest obtiene el mejor desempeño en validación (F1-macro 0,846), por encima de XGBoost y de las tres arquitecturas neuronales. Sin embargo, esta comparación no constituye un experimento de balanceo estrictamente controlado: Regresión Logística y Random Forest utilizaron `class_weight="balanced"`, XGBoost no incorporó ningún tratamiento del desbalance y los MLP se entrenaron con pesos de clase (`class_weight` de Keras). En consecuencia, la tabla permite seleccionar una familia prometedora, pero no atribuir la diferencia observada exclusivamente al algoritmo.

3.3.1. Validación bajo un protocolo de balanceo homogéneo

Para controlar esta variable de confusión, se reejecutó la comparación aplicando SMOTE de manera uniforme —ajustado únicamente sobre el conjunto de entrenamiento— a las cinco técnicas cuya representación de entrada es compatible con el remuestreo sintético: Regresión Logística, Random Forest, XGBoost, MLP Shallow y MLP Profunda. Se eliminó cualquier otro mecanismo de balanceo (`class_weight`) de estos cinco modelos, de modo que la única fuente de tratamiento del desbalance sea la misma para todos.

La arquitectura MLP con Embeddings se excluye deliberadamente de este protocolo homogéneo: a diferencia de las demás técnicas, sus variables categóricas se representan como índices enteros (para alimentar las capas de embedding) y no como vectores one-hot. SMOTE genera vecinos sintéticos por interpolación numérica, lo que produciría índices de categoría fraccionarios y sin significado categórico válido. Mantener la arquitectura de embeddings fuera de esta comparación específica es, por tanto, una decisión metodológica justificada y no una omisión.

Modelo	Accuracy	F1-macro
Regresión Logística	0,767	0,759
Random Forest	0,860	0,848
XGBoost	0,846	0,831
MLP Shallow	0,806	0,797
MLP Profunda	0,803	0,796

Tabla 5. Comparación homogénea: SMOTE aplicado únicamente sobre el conjunto de entrenamiento, para las cinco técnicas comparables.

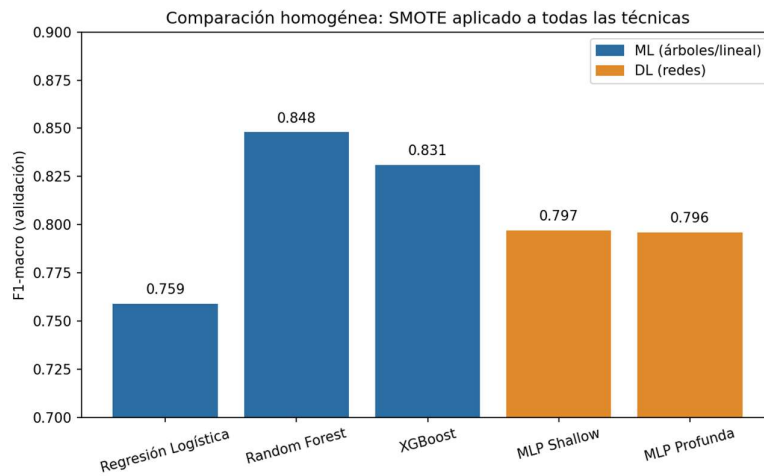


Figura 3. Comparación homogénea de F1-macro (SMOTE uniforme, sin `class_weight`).

El resultado bajo protocolo controlado reproduce el mismo ordenamiento: Random Forest se mantiene como la técnica de mejor desempeño (F1-macro 0,848), seguida de XGBoost (0,831) y de las arquitecturas MLP (0,797 y 0,796). La magnitud de las diferencias respecto de la comparación original es mínima (variaciones de 0,000 a 0,008 en F1-macro), lo que permite concluir que la superioridad observada de Random Forest no era un artefacto del tratamiento desigual del desbalance, sino un resultado robusto frente a un protocolo de comparación equivalente entre familias de modelos.

Restricciones computacionales. Los modelos clásicos admiten entrenamiento paralelo en CPU, mientras los MLP requieren TensorFlow, mayor memoria para matrices densas y entrenamiento por épocas. Los tiempos de las arquitecturas neuronales continúan sin instrumentarse de forma comparable entre sí y respecto de los modelos clásicos; esta limitación se mantiene y se documenta en la sección V.

3.4. Requisitos del proyecto

A partir del alcance definido se identifican los siguientes requisitos, vinculados explícitamente a los objetivos. La explicabilidad se satisface mediante atribuciones SHAP locales calculadas sobre el modelo final real (ver sección 3.8).

Tipo	Requisito	Cómo se satisface
Funcional	Clasificar el segmento de precio	Modelo RF + SMOTE (sin age) expuesto vía API
Funcional	Entregar probabilidades por clase	Salida predict_proba en /predict
No funcional	Usabilidad	Interfaz web con formulario guiado
No funcional	Explicabilidad	SHAP local sobre el modelo final real, en cada predicción
No funcional	Escalabilidad	Backend FastAPI, modelo serializado
No funcional	Seguridad	Validación de entradas con Pydantic
No funcional	Confiabilidad	Partición estratificada de entrenamiento, validación y prueba
No funcional	Monitoreabilidad	Endpoint /metrics con uso y latencia

Tabla 6. Tipo, requisitos y su satisfacción.

3.5. Selección de la arquitectura

Siguiendo los fundamentos de Goodfellow et al. (2016), se evaluaron tres arquitecturas de perceptrón multicapa, todas con salida softmax de tres neuronas (una por clase) y entrenadas con la función de pérdida de entropía cruzada categórica y el optimizador Adam. Se aplicó early stopping para evitar el sobreajuste.

- **MLP Shallow:** una capa oculta de 64 neuronas con activación ReLU. Baseline neuronal.
- **MLP Profunda:** tres capas ocultas (128-64-32) con normalización por lotes (BatchNorm) y regularización por Dropout (0,3).
- **MLP con Embeddings:** capas de embedding para las variables categóricas de alta cardinalidad (marca, tipo), concatenadas con las numéricas antes de las capas densas.

El análisis de convergencia (Figura 4) muestra que las tres arquitecturas alcanzan una meseta en pocas épocas. La red con embeddings obtuvo el mayor F1-macro entre las MLP (0,801), aunque la diferencia es pequeña.

Cada una de las tres arquitecturas se entrena utilizando su propia instancia de EarlyStopping, creada mediante `create_early_stopping()` en cada llamada a `fit`. Esto evita que el callback conserve el mejor `val_loss` observado durante el entrenamiento de una red anterior, de modo que la época de detención de cada modelo depende únicamente de su propia curva de validación.

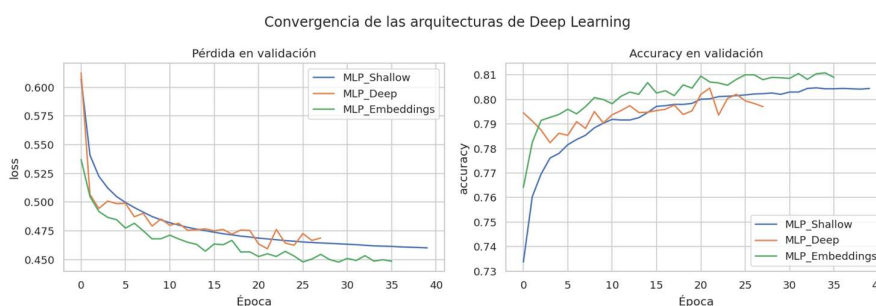


Figura 4. Convergencia de las tres arquitecturas de Deep Learning en el conjunto de validación.

3.6. Refinamiento de Random Forest y balanceo

Para respaldar la decisión de diseño se realizó un barrido de hiperparámetros sobre el Random Forest base con `class_weight="balanced"`, variando `n_estimators` y `max_depth` sobre la misma partición de validación. El experimento se ejecutó en Windows, utilizando Python 3.12.10 y procesamiento en CPU sobre un Intel Core Ultra 7 155U (12 núcleos, 14 procesadores lógicos, 15,5 GB de RAM). Los tiempos son referenciales del entorno utilizado.

Árboles	Profundidad	F1-macro	Recall Premium	Recall Económico	Tiempo (s)	Tamaño (MB)
100	20	0,836	0,855	0,763	6,6	20,1
100	Sin límite	0,846	0,835	0,732	8,8	28,4
200	20	0,839	0,856	0,768	13,3	40,3
200	Sin límite	0,846	0,837	0,729	15,9	56,7
300	20	0,839	0,857	0,767	16,7	60,6
300	Sin límite	0,846	0,835	0,727	21,3	85,2

Tabla 7. Barrido de hiperparámetros del Random Forest base en validación.

El barrido muestra que aumentar de 100 a 200 o 300 árboles sin limitar la profundidad no mejora el F1-macro a tres decimales, que se mantiene en 0,846, pero incrementa progresivamente el tiempo de entrenamiento y el tamaño serializado. El modelo final conserva 200 árboles para mantener la reproducción exacta de los resultados previamente reportados.

El conjunto se particionó de forma estratificada en 70 % entrenamiento, 15 % validación y 15 % prueba, preservando la proporción de clases en cada subconjunto. Dado el desbalance (ratio 3,4× entre la clase mayoritaria y la minoritaria), se analizó el efecto de tres técnicas de balanceo sobre el Random Forest, conforme a las recomendaciones de Chawla et al. (2002) y Chen et al. (2024).

Técnica	F1-macro	Recall Premium	Recall Económico
Sin balanceo	0,846	0,842	0,733
Class weights	0,844	0,832	0,726
SMOTE	0,847	0,865	0,773
Undersampling	0,811	0,927	0,844

Tabla 8. Técnicas de balanceo, F1-macro, Recall Premium y Recall Económico.

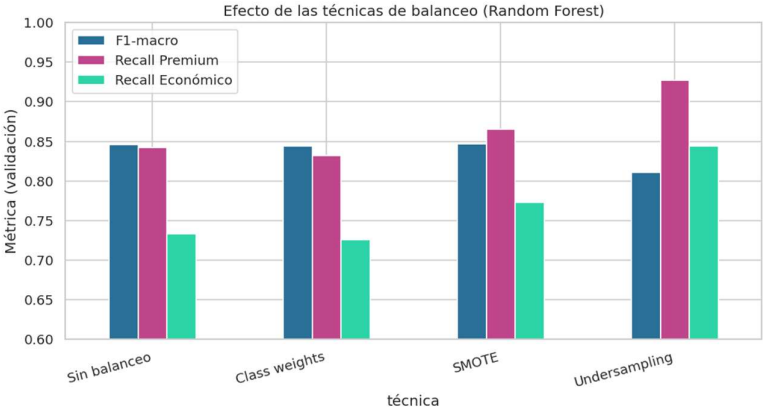


Figura 5. Efecto de las técnicas de balanceo sobre el desempeño del Random Forest.

El sobremuestreo sintético SMOTE (Chawla et al., 2002; Elreedy et al., 2024) ofrece el mejor compromiso: incrementa el recall de las clases minoritarias y mantiene el F1-macro respecto del modelo sin balanceo. El submuestreo maximiza el recall de la clase Premium (0,927) pero degrada el F1-macro al descartar información de la clase mayoritaria. En consecuencia, el modelo final se define como Random Forest combinado con SMOTE.

3.7. Eliminación de la variable redundante age

Dado que age se calcula directamente como 2019 – model_year, ambas variables contienen la misma información. Esta redundancia puede distorsionar la interpretación de la importancia de variables en un análisis SHAP, ya que el crédito de un único hallazgo relacionado con la antigüedad del vehículo se reparte artificialmente entre dos columnas equivalentes. Para evitar este efecto, se evaluó el impacto de eliminar age del conjunto de variables del modelo final.

Para verificarlo, se reentrenó el pipeline final (Random Forest + SMOTE) eliminando age del conjunto de variables numéricas y conservando únicamente model_year, y se comparó contra el modelo original sobre el mismo conjunto de test.

Variante	Accuracy	F1-macro	AUC-OVR macro
Con age y model_year (original)	0,861	0,850	0,954
Sin age (solo model_year)	0,860	0,849	0,953

Tabla 9. Efecto de eliminar la variable redundante age sobre el desempeño del modelo final.

La diferencia de desempeño es prácticamente nula (variación de 0,001 en F1-macro y en AUC-OVR macro), lo que confirma que age no aportaba información adicional a la ya contenida en model_year y que puede eliminarse sin costo real de desempeño. A partir de este resultado, el modelo final del proyecto se redefine excluyendo age del conjunto de variables. Esta decisión, además de evitar la distorsión de importancia en el análisis SHAP (ver sección IV), simplifica el pipeline y reduce el número de variables que la interfaz debe explicar al usuario.

3.8. Desarrollo del frontend y backend

La solución se operacionaliza mediante un backend FastAPI y una interfaz web. El endpoint /predict valida las entradas con Pydantic, construye las mismas características esperadas por el pipeline final (sin age) y devuelve el segmento predicho junto con sus probabilidades. Los endpoints /health y /metrics informan disponibilidad, número de predicciones, distribución de clases y latencias media y p95.

Explicación local de cada predicción. La interfaz no presenta reglas descriptivas fijas ni un análisis de explicabilidad calculado sobre un modelo distinto: cada respuesta del endpoint /predict calcula, en el momento de la solicitud, los valores SHAP de esa predicción específica sobre el mismo Random Forest que la generó (el objeto serializado en models/final_model.joblib). El explicador de SHAP (shap.TreeExplainer) se construye una única vez al iniciar la aplicación, sobre el clasificador ya entrenado, para no reconstruir la estructura de árboles en cada solicitud. La función build_shap_explanation() transforma la entrada con el mismo preprocesador del pipeline, calcula la contribución de cada variable a la clase predicha y devuelve las cinco variables con mayor magnitud de contribución, indicando si empujan la predicción a favor o en contra del segmento asignado.

Esta explicación se probó de extremo a extremo con el cliente de pruebas de FastAPI: para un vehículo Ford tipo truck, año 2019, 25.000 millas, 8 cilindros y tracción 4x4, el modelo predice el segmento Premium con probabilidad 1,0, y la explicación local identifica como principales contribuciones a favor el año del modelo (+0,240), el kilometraje (+0,168), la tracción 4x4 (+0,062), la cantidad de cilindros (+0,060) y la carrocería tipo truck (+0,056) —coherente con la importancia global reportada en la sección IV—.

Esta fidelidad explicativa tiene un costo medible de latencia: la explicación SHAP local añade aproximadamente 2,6 segundos por solicitud, debido al cálculo de valores de Shapley sobre un bosque de 200 árboles. Este trade-off entre fidelidad explicativa y tiempo de respuesta se documenta como limitación abierta en la sección V, con una propuesta concreta de mitigación en la sección de mejoras futuras.

El panel de explicación de la interfaz (frontend/index.html) se denomina “Explicación local de la predicción (SHAP)”, y cada elemento de la lista muestra el nombre de la variable, la dirección de su efecto (a favor o en contra del segmento predicho) y el valor SHAP asociado, dejando explícito que las contribuciones provienen directamente del modelo final.

Tasador de segmento - Vehículos usados

Clasifica un vehículo en Económico, Medio o Premium según sus atributos.

DATOS DEL VEHÍCULO

Marca

ford

Carrocería

truck

Año

2019

Odómetro (millas)

25000

Cilindros

8

Condición

excellent

Combustible

gas

Transmisión

automatic

Color

black

Días de publicación

20

Trazacción 4x4

Si

Clasificar segmento

RESULTADO

Premium

Probabilidad estimada 98.0% - respuesta en 147.97 ms

Económico

0.0%

Medio

2.0%

Premium

98.0%

ORIENTACIÓN SOBRE LOS ATRIBUTOS

- Año reciente: atributo generalmente asociado con segmentos de mayor precio.
- Kilometraje bajo: factor habitualmente asociado con un mayor valor.
- Trazacción 4x4: característica asociada con un mayor valor en el conjunto analizado.
- Motor de ocho o más cilindros: característica frecuente en segmentos de mayor precio.
- Carrocería truck/pickup: categoría asociada con segmentos de mayor precio en este mercado.

Estas observaciones son reglas descriptivas y no atribuciones SHAP locales de esta predicción. Las probabilidades presentadas no han sido calibradas.

MONITOREO DEL MODELO EN PRODUCCIÓN

1

PREDICCIONES

147.97

LATENCIA MEDIA (MS)

147.97

LATENCIA PM (MS)

Económico: 0

Medio: 0

Premium: 1

Fase 3 — Modelo Random Forest + SMOTE (F1-macro 0.848 en test) - Probabilidades no calibradas - Orientación heurística.

Figura 6. Interfaz general del sistema: formulario de ingreso, resultado de la predicción y panel de monitoreo.

Página 15

IV. Resultados

El modelo final, compuesto por Random Forest y SMOTE, sin la variable redundante age (ver sección 3.7), se evaluó sobre un conjunto de prueba independiente de 7.541 vehículos que no fueron utilizados durante el entrenamiento, la validación ni la selección de hiperparámetros. La evaluación considera accuracy, precisión, recall, F1-score y AUC multiclase One-vs-Rest con promedio macro. Los resultados se presentan en la Tabla 10.

Segmento	Precisión	Recall	F1-score	Soporte	AUC-OVR macro
Económico	0,797	0,791	0,794	1.762	—
Medio	0,883	0,883	0,883	4.479	—
Premium	0,866	0,875	0,871	1.300	—
Global (macro)	0,849	0,850	0,849	—	0,953

Tabla 10. Resumen de métricas del modelo final (sin age) sobre el conjunto de prueba.

La exactitud global alcanzó un valor de 0,860 y el F1-macro fue de 0,849. La clase Económico continúa presentando el menor recall (0,791), seguida por Medio (0,883) y Premium (0,875).

El AUC multiclase One-vs-Rest con promedio macro alcanzó un valor de 0,953, evidenciando una alta capacidad del modelo para discriminar entre los segmentos Económico, Medio y Premium a través de diferentes umbrales de decisión. Este valor complementa al F1-macro porque evalúa el ordenamiento de las probabilidades estimadas y no solamente la clase finalmente seleccionada.

La matriz de confusión (Figura 7) muestra que la mayor parte de los errores se concentra entre segmentos vecinos: 365 vehículos Económicos fueron clasificados como Medio y 351 vehículos del segmento Medio fueron clasificados como Económico; 172 vehículos Medios fueron clasificados como Premium y 159 vehículos Premium como Medio. Las confusiones entre los segmentos extremos Económico y Premium suman solo siete casos al considerar ambas direcciones (4 y 3, respectivamente), lo que indica que el modelo representa adecuadamente el orden general de los segmentos de precio.

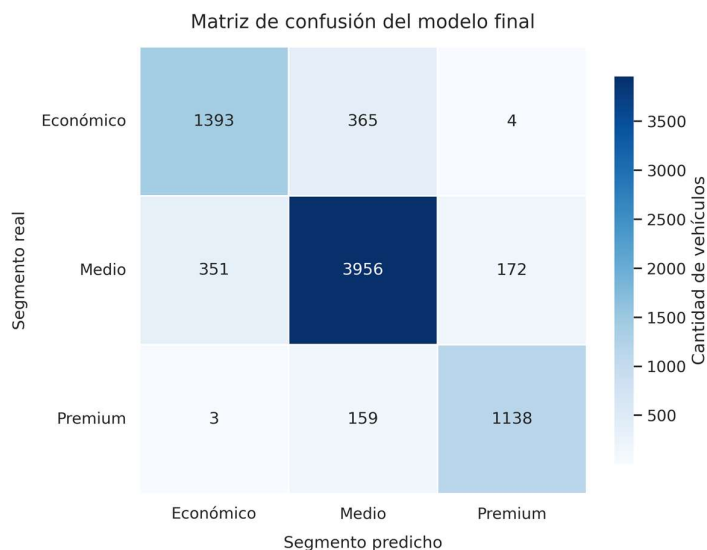


Figura 7. Matriz de confusión del modelo final sobre el conjunto de prueba.

A pesar del AUC elevado, las probabilidades generadas por el modelo no deben interpretarse automáticamente como niveles de confianza perfectamente calibrados; esta limitación permanece abierta

(ver sección V) y su resolución mediante escalado de Platt o calibración isotónica se mantiene como línea de mejora futura.

Interpretabilidad: SHAP sobre el modelo final

El análisis de explicabilidad global se calcula directamente sobre el Random Forest final de 200 árboles, el mismo objeto serializado que expone la API, evitando así cualquier discrepancia entre el modelo analizado y el modelo en producción.

Al no incluir la variable age (sección 3.7), el ranking de importancia no reparte artificialmente el crédito de la antigüedad del vehículo entre dos columnas redundantes: model_year encabeza limpiamente la importancia global, seguida de odometer, is_4wd, cylinders y condition_ord.

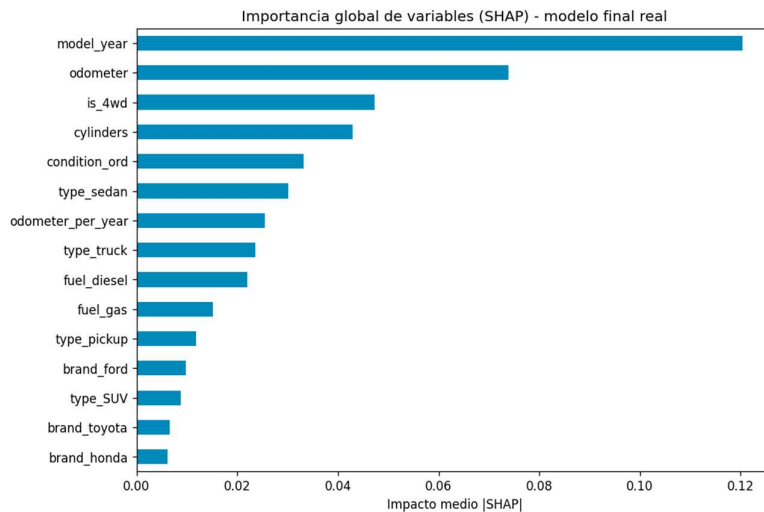


Figura 8. Importancia global de variables según SHAP, calculada sobre el modelo final.

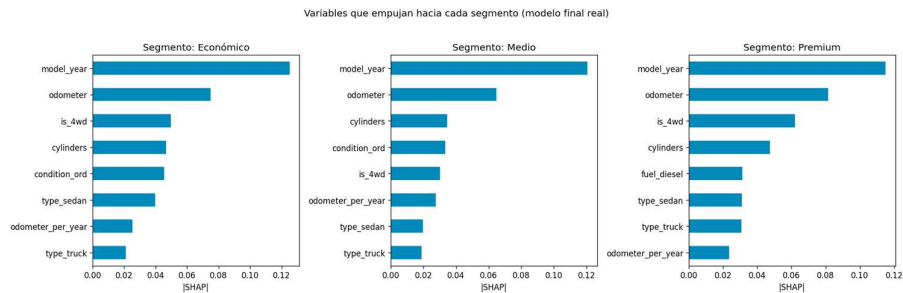


Figura 9. Variables que empujan la clasificación hacia cada segmento de precio, modelo final.

Las asociaciones identificadas mediante SHAP son coherentes con la lógica del mercado automotor — vehículos más nuevos, con menor kilometraje, tracción 4x4 y mayor cilindrada se asocian a segmentos de mayor precio—, pero no deben interpretarse como relaciones causales.

Como se describió en la sección 3.8, cada predicción del endpoint /predict incluye además una explicación local calculada sobre este mismo modelo, de modo que el análisis global (Figuras 8 y 9), la explicación que ve el usuario en la interfaz y el modelo que produce la predicción corresponden todos al mismo objeto, sin sustitutos intermedios.

En conjunto, los resultados muestran que el modelo cumple el objetivo funcional de clasificar vehículos en tres segmentos de precio, mantiene un desempeño equilibrado frente al desbalance de clases —verificado bajo un protocolo de comparación homogéneo entre familias de modelos (sección 3.3.1)— y presenta una

explicabilidad consistente de extremo a extremo, desde el análisis global hasta cada predicción individual. No obstante, estos resultados corresponden exclusivamente al conjunto de datos analizado y no garantizan el mismo comportamiento en otros mercados, periodos o plataformas.

V. Conclusiones, limitaciones y trabajos futuros

5.1. Reflexión crítica sobre el proceso de consolidación

Llegar a esta etapa final del proyecto dejó un aprendizaje que va más allá de lo técnico: identificar una limitación en un informe es relativamente sencillo una vez que se sabe dónde mirar; resolverla con evidencia empírica —y no solo con una explicación textual mejor redactada— exige volver al código, reejecutar experimentos, y aceptar que el resultado de esa verificación podía haber sido negativo.

De las observaciones metodológicas revisadas en esta etapa, dos se confirmaron completamente resueltas con impacto nulo o positivo en el desempeño: la comparación homogénea de balanceo (sección 3.3.1) reprodujo el mismo ordenamiento de técnicas bajo un protocolo controlado, y la eliminación de la variable redundante age (sección 3.7) no costó desempeño medible. La tercera —la coexistencia de distintos niveles de explicabilidad en el proyecto— resultó ser la más costosa de resolver en términos de ingeniería: no bastaba con recalcular SHAP sobre el modelo final, sino que había que rediseñar el flujo de la API para calcular explicaciones locales en tiempo de solicitud, lo que introdujo un costo de latencia que no existía antes. Esto fue, en sí mismo, un aprendizaje relevante del curso: la explicabilidad rigurosa tiene un costo computacional real, y ese costo es parte del diseño de un sistema de aprendizaje automático en producción, no un detalle posterior.

Se verificó también, sin necesidad de modificarlo, que el manejo de EarlyStopping en las arquitecturas de Deep Learning ya utilizaba una instancia independiente por modelo. Este punto deja una lección adicional: no toda revisión crítica implica trabajo nuevo; parte del proceso consiste en auditar el estado real del código antes de asumir que algo debe rehacerse.

El equipo optó deliberadamente por no aumentar la complejidad del sistema (por ejemplo, mediante un ensamble stacking) en esta fase, priorizando consolidar comparabilidad experimental, reproducibilidad y explicabilidad local antes de añadir nuevos componentes, lo que se considera la decisión metodológica correcta dado el estado del proyecto.

5.2. Limitaciones

Las siguientes limitaciones permanecen vigentes y delimitan el alcance de las conclusiones del proyecto:

- El modelo se entrenó con avisos del mercado estadounidense correspondientes a un periodo (mayo de 2018 a abril de 2019) y una plataforma específicos. Sus métricas internas no garantizan el mismo comportamiento en Chile u otros mercados.
- Los umbrales fijos de USD 5.000 y USD 20.000 no se ajustan por inflación, categoría del vehículo ni cambios temporales. La confusión Económico↔Medio observada en la matriz de confusión es consistente con vehículos situados cerca de esos límites.
- Las probabilidades del Random Forest no han sido sometidas a un procedimiento de calibración; un AUC elevado indica alta capacidad de discriminación, pero no garantiza probabilidades calibradas.
- La comparación de técnicas de balanceo (sección 3.6) se realizó únicamente sobre Random Forest; no se replicó el barrido completo de SMOTE, class weights y undersampling para XGBoost ni para las arquitecturas de Deep Learning.
- Los tiempos de entrenamiento de las arquitecturas neuronales no fueron instrumentados de forma comparable con los modelos clásicos.
- El cálculo de SHAP local en tiempo de solicitud es computacionalmente costoso sobre un bosque de 200 árboles sin límite de profundidad: añade aproximadamente 2,6 segundos por predicción. Para

un servicio con mayor volumen de solicitudes concurrentes, esta latencia sería un cuello de botella que debe optimizarse antes de un despliegue productivo real.

5.3. Propuesta de mejoras

Como primera línea de mejora, se propone optimizar la latencia de la explicación local, ya sea mediante un explicador SHAP aproximado (por ejemplo, limitando la profundidad efectiva considerada o usando un muestreo de árboles), mediante la precomputación de estructuras auxiliares del TreeExplainer, o evaluando si una configuración de 100 árboles sin límite de profundidad —que el barrido de la Tabla 7 identificó como la más eficiente en tiempo de entrenamiento— también reduce significativamente el tiempo de explicación sin pérdida relevante de F1-macro.

Como segunda línea de mejora, se recomienda calibrar las probabilidades mediante escalado de Platt o calibración isotónica, de modo que los porcentajes mostrados en la interfaz representen de manera más fiable la frecuencia real de aciertos del modelo, especialmente en predicciones extremas.

Como tercera línea, y solo una vez consolidadas las bases anteriores, se propone evaluar un ensamble stacking de Random Forest, XGBoost y MLP con embeddings, y extender la comparación homogénea de balanceo a XGBoost y a la arquitectura de embeddings mediante un esquema de remuestreo compatible con variables categóricas indexadas (por ejemplo, SMOTENC).

Finalmente, se mantiene la recomendación de enriquecer el conjunto de datos con información sobre equipamiento, historial del vehículo, ubicación y condiciones del mercado, y de evaluar segmentos condicionados por tipo de carrocería.

5.4. Conclusión

El proyecto permitió desarrollar, evaluar y consolidar una solución de aprendizaje automático para clasificar vehículos usados en los segmentos Económico, Medio y Premium. Random Forest combinado con SMOTE, sin la variable redundante age, obtuvo el mejor equilibrio entre las clases, alcanzando una exactitud de 0,860, un F1-macro de 0,849 y un AUC-OVR macro de 0,953 sobre un conjunto de prueba independiente de 7.541 registros.

Más allá de las métricas, el aprendizaje central de esta fase final fue metodológico: un proyecto de aprendizaje automático no se consolida solamente reportando buenos resultados, sino demostrando que esos resultados sobreviven a un escrutinio riguroso de las propias decisiones de diseño. Verificar que Random Forest sigue siendo superior bajo un protocolo de balanceo controlado, comprobar empíricamente que una variable puede eliminarse sin costo, y llevar la explicabilidad hasta el mismo objeto que produce cada predicción, son ejercicios que exigieron volver atrás, cuestionar trabajo ya entregado, y aceptar el costo de ingeniería que eso implica —en este caso, un aumento de latencia a cambio de explicaciones que finalmente son ciertas—.

La integración mediante FastAPI y una interfaz web, con explicabilidad local calculada sobre el modelo real, demuestra la viabilidad de operacionalizar la solución completa: desde el análisis exploratorio de datos hasta una predicción individual explicada matemáticamente sobre el mismo modelo que la generó. Este resultado cumple el objetivo funcional definido para el proyecto y dejó al equipo con una comprensión más realista de las brechas entre un modelo que funciona en una libreta de análisis y un sistema que responde, de forma consistente y explicable, a solicitudes externas.

VI. Código fuente en GitHub

El código completo del proyecto se encuentra disponible en el siguiente repositorio:

https://github.com/niconvc/acif104_s9_equipo4

El repositorio se organiza en las carpetas data (dataset), src (preprocesamiento, entrenamiento de modelos, balanceo y explicabilidad), models (modelo serializado), backend (API FastAPI), frontend (interfaz web), reports (resultados y figuras) y docs (documentación del proyecto). Los siguientes archivos son los relevantes para las mejoras metodológicas descritas en las secciones III y IV:

- **src/homogeneous_comparison.py** (nuevo): reejecuta la comparación ML/DL aplicando SMOTE de forma homogénea sobre las cinco técnicas comparables. Genera reports/homogeneous_comparison_results.csv.
- **src/feature_redundancy_check.py** (nuevo): compara el modelo final con y sin la variable age. Genera reports/feature_redundancy_results.csv.
- **src/final_model.py** (modificado): el pipeline final excluye age del conjunto de variables numéricas. Regenera models/final_model.joblib, reports/final_model_results.csv, reports/confusion_matrix.csv y la figura de matriz de confusión.
- **src/explain.py** (reescrito): calcula SHAP directamente sobre el Random Forest final serializado (ya no entrena un modelo sustituto) y expone explain_instance(), reutilizada por la API para explicaciones locales. Regenera reports/figures/shap_global.png y reports/figures/shap_por_clase.png.
- **backend/app.py** (modificado): construye un explicador SHAP una sola vez al iniciar el servicio y reemplaza build_heuristic_explanation() por build_shap_explanation(), que calcula la atribución local real para cada predicción del endpoint /predict. El campo explanation_type cambia de "heuristic" a "shap_local".
- **frontend/index.html** (modificado): actualiza el panel de explicación para mostrar las contribuciones SHAP locales (variable, dirección del efecto y valor SHAP) en vez de las reglas descriptivas fijas.

El modelo serializado (models/final_model.joblib) se excluye del repositorio mediante .gitignore por su tamaño (66,3 MB) y debe regenerarse localmente ejecutando, en orden, desde la raíz del repositorio:

- python src/final_model.py
- python src/explain.py
- python src/homogeneous_comparison.py (opcional, evidencia de la sección 3.3.1)
- python src/feature_redundancy_check.py (opcional, evidencia de la sección 3.7)

El repositorio conserva además los scripts reproducibles de las fases previas del proyecto (preprocessing.py, eda.py, train_ml.py, train_dl.py, balancing.py, rf_refinement.py).

VII. Bibliografía

- Bergmann, S., y Feuerriegel, S. (2025). Machine learning for predicting used car resale prices using granular vehicle equipment information. *Expert Systems with Applications*, 263, 125640. <https://doi.org/10.1016/j.eswa.2024.125640>
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- Chawla, N. V., Bowyer, K. W., Hall, L. O., y Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357. <https://doi.org/10.1613/jair.953>
- Chen, T., y Guestrin, C. (2016). XGBoost: A scalable tree boosting system. En *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). <https://doi.org/10.1145/2939672.2939785>
- Chen, W., Yang, K., Yu, Z., Shi, Y., y Chen, C. L. P. (2024). A survey on imbalanced learning: Latest research, applications and future directions. *Artificial Intelligence Review*, 57(6), 137. <https://doi.org/10.1007/s10462-024-10759-6>
- Elreedy, D., Atiya, A. F., y Kamalov, F. (2024). A theoretical distribution analysis of synthetic minority oversampling technique (SMOTE) for imbalanced learning. *Machine Learning*, 113(7), 4903–4923. <https://doi.org/10.1007/s10994-022-06296-4>
- Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (3.^a ed.). O'Reilly Media.
- Goodfellow, I., Bengio, Y., y Courville, A. (2016). *Deep learning*. MIT Press.
- Grinsztajn, L., Oyallon, E., y Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? *Advances in Neural Information Processing Systems*, 35, 507–520.
- Li, C. (2024). Machine learning-based models for accurate car prices prediction. *Highlights in Business, Economics and Management*, 40. <https://doi.org/10.54097/9zcpv779>
- Lundberg, S. M., y Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.
- Murphy, K. P. (2018). *Machine learning: A probabilistic perspective*. MIT Press.
- Qian, T. (2023). Used car price prediction by using XGBoost. *BCP Business & Management*, 44. <https://doi.org/10.54691/bcpbm.v44i.4794>
- Shwartz-Ziv, R., y Armon, A. (2022). Tabular data: Deep learning is not all you need. *Information Fusion*, 81, 84–90. <https://doi.org/10.1016/j.inffus.2021.11.011>